

florin-parser-nano: Fine-Tuning a 1.2B Open-Weight Vision– Language Model for Insurance Document Parsing on ParseBench

ammoman21

Florin

September 1, 2026

Abstract

Automated insurance underwriting requires parsing large volumes of visually complex documents—rate filings, loss runs, policy forms—into structured, machine-readable text. On ParseBench, a public document-parsing benchmark whose Tables dimension is 57.7% insurance documents, the leading open-weight parser scores 3.73 points lower on insurance tables than on the non-insurance remainder, indicating that this document class is measurably harder than the corpus average for existing systems. We study whether a small open-weight model can be improved on this benchmark through failure-driven fine-tuning. Starting from KDL-Frontier-Parser-nano (1.2B parameters), we first reproduce the published baseline in our own environment, diagnose its dominant failure mode (the model emits no inline formatting markup at all, and prompting cannot fix this), construct 6,678 training pairs from SEC EDGAR insurance-carrier filings and synthetically rendered pages, and train a low-rank adapter on the language layers only, leaving the production prompt byte-identical. The resulting pipeline, *florin-parser-nano*, scores **76.69** Overall (mean of two full-corpus runs, 76.72 / 76.66)—the highest score of any vision–language model on the leaderboard, open-weight or proprietary, and fifth overall behind four commercial entries. The same-environment improvement over our reproduction of the previous open-weight leader is +4.04 points. Alongside the entry we disclose five benchmark defects (filed upstream as issues #100–#104), and we document a maintainer-review correction in which two defects in our own emission code were found, fixed, and re-scored on identical model output the same day, revising our score from 76.92 to 76.69. All claims were preregistered before measurement; this is a benchmark study, and the model is not deployed in any production system.

1 Introduction

Florin is building a full-stack insurance company that operates with no human underwriters; central to that thesis is software capable of autonomous underwriting. A prerequisite for any such system is document parsing at scale: underwriting decisions consume rate filings submitted through SERFF (the System for Electronic Rate and Form Filing, the channel through which United States insurance carriers file rates with state regulators), loss-run reports, policy forms, actuarial memoranda, and insurer financial statements. These documents are dominated by dense, deeply nested tables—rate relativity grids, per-policy rate tables, merged cells spanning rating dimensions—and by meaning-bearing typography: struck-through text marks superseded terms, and superscripts are footnote references that change how a figure reads. A parser that silently drops them returns text that is complete and misleading.

Public evidence suggests existing systems handle this document class worse than average enterprise documents. ParseBench^[1], a live document-parsing benchmark maintained by LlamaIndex, draws 57.7% of its Tables dimension (290 of 503 pages) from insurance documents, and our measurement of the benchmark's leading open-weight parser shows it scoring **84.18** on those insurance tables versus **87.91** on the non-insurance

remainder—a 3.73-point gap on the largest, most reliable subset in the corpus (Section 2.2). The published Tables scores on this benchmark are therefore being carried upward by the non-insurance minority, not the reverse.

Motivated by this gap, we investigate how far a small open-weight model can be pushed on this benchmark by a disciplined loop of baseline reproduction, failure diagnosis, targeted data construction, and parameter-efficient fine-tuning. Our contributions are:

- A fine-tuned 1.2B-parameter open-weight parser, *florin-parser-nano*, scoring 76.69 Overall on ParseBench—the top score in the benchmark’s vision–language-model (VLM) category, ahead of both the best proprietary VLM (Gemini 3 Flash, 75.05) and the previous open-weight leader (76.36)—with weights, training code, and data generators public.
- A same-environment, preregistered evaluation protocol in which every improvement is measured against our own reproduction of the baseline (72.65), not against published numbers we could not reproduce.
- Five benchmark and scorer defects disclosed to the maintainers (issues #100–#104^[7]), plus a documented correction episode in which maintainer review found two defects in our own emission code; both were fixed and re-scored on identical model output the same day (Section 5).

Consistent with the public preregistration^[5], this work is a benchmark study conducted for publication. The model was not built for, and is not deployed in, a production system.

2 Background: ParseBench

2.1 What the benchmark measures

ParseBench scores document parsers over 2,079 annotated pages of enterprise documents using 169,011 deterministic rule checks. A parser converts each page image (or PDF) to markdown; the markdown is graded against per-page annotations across five dimensions, and the Overall score is the unweighted mean of the five. Because grading is rule-based (with one optional LLM-judge component discussed in Section 5), runs are reproducible and inexpensive. Submission is self-serve: a pull request adds a results file and provider code to the benchmark repository, and maintainers review and re-run entries. At the time of writing the leaderboard holds 88 entries: 57 vision–language models (31 open-weight, 26 proprietary), 22 commercial parsing products, and 9 local open-source tools.

Table 1: The five ParseBench dimensions, their metrics, and corpus composition.

Dimension	Metric	Pages	Insurance pages	Insurance share
Tables	grits_trm_composite	503	290	57.7%
Charts	rule_pass_rate	568	40	7.0%
Content Faithfulness	content_faithfulness	506	25	4.9%
Semantic Formatting	semantic_formatting	476	24	5.0%
Visual Grounding	layout_element_rule_pass_rate	500	29	5.8%

Tables: GriTS (grid table similarity, a structural match over rows and columns) composited with a record-matching term. Charts: annotated data points must be recovered with correct labels. Content Faithfulness: omissions, hallucinations, and reading-order violations at word, sentence, and digit granularity (141,322 rules). Semantic Formatting: preservation of meaning-bearing inline styling—bold, strikethrough, super/subscript, headings, LaTeX, code fences. Visual Grounding:

localization, element classification, and attribution of each extracted element to its page region. Page counts and insurance counts from the subset analysis in the reproduction repository (`reports/insurance_subset_scores.md`); some pages appear in more than one dimension.

2.2 Corpus composition: an insurance-heavy benchmark

Three independent passes over the corpus—filenames, all 169,011 ground-truth rule strings, and the text layer of the PDFs—identify **384 of 2,079 documents (18.5%) as insurance**, with the concentration in exactly the dimension that matters most for document extraction: the Tables dimension is majority insurance (193 SERFF rate-filing pages, 78 further pages from auto and farm property-casualty filings, plus Solvency II reports and life-insurer financial statements). On our full-corpus baseline run, the leading open-weight parser scored 84.18 on the 290 insurance table pages versus 87.91 on the 213 non-insurance table pages (−3.73). This is the one insurance-subset comparison large enough to support a confident claim; the other four dimensions' insurance slices are 24–40 pages and we treat them as directional only.

3 Method

3.1 Baseline reproduction and parity discipline

The base model is `KDLAI/KDL-Frontier-Parser-nano`^[3] (KoreaDeepLearning; 1.2B parameters, Qwen2-VL architecture, AGPL-3.0), which held the top open-weight position with a published Overall of 76.36. Before changing anything we reproduced its pipeline in our own environment: vLLM serving on a single NVIDIA H100 80GB with the pipeline's own documented flags, the complete 2,079-file corpus, the benchmark's own evaluation code and default metric per dimension, and the chart LLM normalizer disabled and declared (`LLAMACLOUD_BENCH_LLM_NORMALIZATION=off`; see Section 5).

Table 2: Parity check—our reproduction of the baseline vs. its published scores.

Dimension	Ours	Published	Δ	Status
Tables	85.76	85.56	+0.20	reproduced
Charts	63.69	63.41	+0.28	reproduced
Content Faithfulness	87.18	87.19	−0.01	reproduced
Semantic Formatting	52.42	66.81	−14.39	not reproduced
Visual Grounding	74.19	78.84	−4.65	not reproduced
Overall	72.65	76.36	−3.71	

Three of five dimensions reproduce within 0.3 points; two do not, and we could not explain the shortfall. A control pipeline (`Chandra-ocr-2`) run through the identical harness reproduced its own published Charts score exactly (65.10) and its Semantic Formatting within 3.17, establishing that the harness and scoring are sound and the deficits are specific to the base pipeline's behavior in our environment. Two candidate causes were tested and eliminated: the optional chart LLM normalizer (scoped to chart metrics only, and our Charts matched) and output truncation (zero length-terminated generations across 1,200 saved outputs). The consequence for claims is fixed in the preregistration: **every improvement is measured against our own 72.65 reproduction in the same environment**; the comparison to the published 76.36 crosses evaluation environments and is always reported with that caveat.

3.2 Failure diagnosis

Across 23,802 recognized elements from 1,196 documents, the base pipeline emits **zero** inline formatting markup—no bold, strikethrough, superscript, or subscript markers of any kind. Its instruction set is five bare prompt strings (e.g. "`\nText Recognition:\n`") with no formatting request anywhere, so 100% of its Semantic Formatting bold credit arrives through an indirect scorer path (headings render bold; see Section 5). Of the scored bold failures, 63.3% are text merged inline into a longer output line: the characters are present but not separable, so no post-processing can reach them—only the model marking spans at generation time can.

Prompting cannot supply this behavior. Four prompt variants were measured on a 40-document development subset against the byte-exact production prompt as control: the model *can* emit markers when asked (1,729 markers under the most explicit variant, versus zero under control), but it marks the wrong spans, and its bold sub-score degrades monotonically with the amount of prompt perturbation ($60.59 \rightarrow 58.96 \rightarrow 56.94 \rightarrow 55.24$). All four variants scored at or below control Overall. Two design decisions follow directly: the thing to train is span-level *judgment*, not marker syntax; and training must leave the production prompt byte-identical, because the model is brittle to any prompt change.

3.3 Training data: EDGAR fragments and synthetic rendering

We built 6,678 region-crop→markdown training pairs from two sources, neither of which touches ParseBench's annotations (which are the test set; using them would be contamination):

- **2,165 real fragments from SEC EDGAR** (the U.S. Securities and Exchange Commission's public filing system^[8]) insurance-carrier filings. Because EDGAR filings exist as HTML, bold ground truth is derived mechanically from the document object model rather than annotated by hand.
- **3,313 synthetic rendered fragments**: business-document pages generated in HTML/CSS with known bold runs, strikethrough amendments, and superscript/subscript footnote references, rendered to images at pipeline resolution—so the target markdown is exact by construction. This is the sole source of strikethrough and super/subscript supervision, and includes CJK newsprint and Devanagari textures to match the corpus's multilingual pages.
- **1,200 no-styling negatives** whose correct output contains no markers, to teach restraint: the measured failure mode of explicit prompting was indiscriminate marking, and without negatives a fine-tune reproduces it.

3.4 Fine-tuning configuration

We trained a LoRA adapter (low-rank adaptation^[9]: the base weights stay frozen and small low-rank correction matrices are trained) with rank 16 on the language model's attention and MLP layers only; the vision tower and the vision–language projector are frozen. The loss is completion-only (computed on the target markdown, not the prompt), and the prompt is byte-identical to the production pipeline's "`\nText Recognition:\n`". Training ran 2 epochs at learning rate $5e-5$ in bfloat16 on a single H100, roughly 50 minutes per run. The full campaign was 7 training iterations selected on the 40-document development subset (never on the benchmark's published leaderboard), about 9 hours and \$31 of GPU time end to end, followed by the two preregistered full-corpus runs.

3.5 Emission layer

Every VLM entry on ParseBench is scored on markdown produced jointly by the model and declared provider code in the benchmark repository—the code that converts the model's per-element output (category, bounding box, text) into the final markdown string. Our entry is no exception, and we state its emission layer plainly: the scored output is the fine-tuned model *plus* the provider `florin_parser_nano.py` submitted in PR #99^[6], which carries three emission changes relative to the base pipeline's provider, each preregistered as a defect fix or a real inference the pipeline was not making:

- heading levels derived from title-element bounding-box height (the base pipeline performs no hierarchy inference; every heading collapses to a single level);
- bold applied to run-in `Label`: prefixes, which are bold in the source documents and dropped by the base pipeline;
- a relaxed heading gate that stops rejecting genuine headings ending in a colon, with a preregistered 20-word length cap.

A fourth preregistered fix (a missing `section_header` key in the base pipeline's category map, dead code confirmed by zero occurrences across 2,078 artifacts) ships in the provider but is excluded from all arithmetic, because replay cannot measure its contribution. The emission patches without any fine-tuning were measured by replay at 74.64 Overall (+1.99 over the 72.65 reproduction; a pre-correction replay figure, re-measurement pending)—short of the published leader, which is why the fine-tune, not the emission layer, is the substance of this entry. The final emission code is exactly the version corrected after maintainer review (Section 5.2).

3.6 Preregistration

Before any measured run we committed a public preregistration^[5] fixing the base model, serving flags, corpus, metrics, run protocol (one validation run, one frozen confirmation run, tolerance 0.3 Overall between them), kill criteria (any change costing more than 0.3 on Tables, Content Faithfulness, or Visual Grounding is dropped regardless of gains elsewhere), and three claim tiers tied to score thresholds. One dated amendment corrected a mischaracterization in the original document before any live run and re-measured the affected figures; the amendment is preserved verbatim alongside the original text.

4 Results

All numbers in this section are the corrected values after the maintainer-review episode of Section 5.2; the pre-correction values are reported there.

Table 3: ParseBench scores of florin-parser-nano (two full-corpus runs and their mean) against the base model's published row and our same-environment reproduction of it.

Dimension	Run 1	Run 2	Mean	Base (published)	Base (same env.)
Tables	86.14	86.05	86.10	85.56	85.76
Charts	65.25	65.13	65.19	63.41	63.69
Content Faithfulness	87.38	87.36	87.37	87.19	87.18
Semantic Formatting	70.66	70.62	70.64	66.81	52.42
Visual Grounding	74.15	74.14	74.14	78.84	74.19
Overall	76.72	76.66	76.69	76.36	72.65

Same-environment improvement: +4.04 Overall (72.65 $\rightarrow$ 76.69), the comparison we consider primary because both sides use identical serving, corpus, and scoring. The gain decomposes per dimension as Semantic Formatting +18.22, Charts +1.50, Tables +0.34, Content Faithfulness +0.19, Visual Grounding -0.05 . Against the base model's published row the margin is +0.33, reported with the standing caveat that we could not reproduce that published number (Section 3.1) and that the maintainers' own re-run governs if it lands elsewhere.

Two-run agreement. The two independent full-corpus runs differ by 0.06 Overall with a maximum per-dimension difference of 0.12—within the preregistered 0.3 tolerance, so the mean is reported as specified in advance.

4.1 Leaderboard position

Table 4: Top of the ParseBench leaderboard at submission (from the benchmark's own top-10 table). ¢/page as published; open-weight models are self-hosted.

Rank	Entry	Category	Overall	¢ / page
1	LlamaParse Agentic Plus	Commercial (LlamaParse)	90.20	5.62
2	LlamaParse Agentic	Commercial (LlamaParse)	87.01	1.25
3	LlamaParse Cost Effective	Commercial (LlamaParse)	80.61	0.38
4	Pulse Ultra 2	Commercial (startup API)	77.08	15.00
5	florin-parser-nano (ours)	VLM – Open Weight	76.69	—
6	KDL-Frontier-Parser-nano	VLM – Open Weight	76.36	—
7	Extend (2.0)	Commercial (startup API)	75.33	2.50
8	Google Gemini 3 Flash (Thinking High)	VLM – Proprietary	75.05	2.41
9	Infinity-Parser2-Pro	VLM – Open Weight	74.28	—
10	Extend Light (1.0)	Commercial (startup API)	73.26	0.62

florin-parser-nano is, at submission: **the highest-scoring vision–language model on the board of any kind**, open-weight or proprietary—the best proprietary VLM is Gemini 3 Flash (Thinking High) at 75.05, 1.64 points behind; **the highest-scoring open-weight entry**, 0.33 ahead of the base model it was fine-tuned from (76.36); and **fifth overall**, behind four commercial product entries—three configurations of LlamaParse (90.20, 87.01, 80.61), which is the benchmark maintainers' own product line, and Pulse Ultra 2 (77.08). Its Semantic Formatting score of 70.64 also exceeds the best other open-weight formatting entry on the board (69.30).

5 Findings: benchmark defects and the correction episode

5.1 Five defects disclosed upstream (issues #100–#104)

Working at parity depth surfaced defects in the benchmark itself, filed as GitHub issues against the benchmark repository^[7]:

- **#100 — Chart rule rejects correct extraction on typographic grounds.** The chart data-point scorer only credits a chart-title label found in surrounding text if that text is bold, a heading, or a caption; a parser that extracts every data point correctly but renders the title as plain text loses the points.
- **#101 — Zero-weight annotation categories, undocumented.** `is_italic` (655 rules), `is_underline` (405), and `is_mark` (88) are annotated in the dataset and evaluated, but contribute to no scored category—the styling sub-score is built from bold, strikethrough, superscript, and subscript only.
- **#102 — Silent default layout adapter.** For a provider key with no registered layout adapter, the registry silently returns a default adapter instead of raising, which makes the intended shape-matching fallback unreachable; a new pipeline can score approximately zero on Visual Grounding with no registry-level error anywhere.
- **#103 — LLM-judge normalization on by default, contradicting the documentation.** Chart evaluation applies a Claude-Haiku LLM normalization pass unless an environment variable is explicitly set to off; the README describes evaluation as deterministic and two docstrings state the default is off. (All runs in this paper declare it off.)
- **#104 — Bold-heading coupling.** The bold matcher accepts any heading line containing the annotated text as evidence of bold. Because the base pipeline emits no inline markup, 100% of its bold credit arrives through this route, and the dataset contains zero negative styling rules, so indiscriminate promotion costs nothing inside the dimension. By replay, a markdown-only transformation set containing no bold detection at all measured 75.65 Overall—within 0.71 points of the then-leading published score. We disclosed this instead of submitting it; the measurement and reasoning are in the preregistration's amendment.

5.2 Maintainer review found two defects in our emission code—the correction

We report this episode in full because it is evidence the process works. Our entry was submitted at Overall 76.92 (run means 76.95 / 76.89). The ParseBench maintainer's review of PR #99 found two defects in our markdown emission that inflated Semantic Formatting: (1) the run-in-label bolder matched against the whole line, so a list item - Note: x became **- Note:** x**—the bullet swallowed into the bold span, the list broken—and, lacking fence-state tracking, it bolded label-shaped lines inside code/LaTeX fences; because the scorer's bold matcher searches raw text, the corrupted spans still earned credit; (2) our relaxed heading gate had dropped the stock terminal-punctuation veto entirely, promoting short sentences ending in a period to headings.

Both defects were reproduced byte-for-byte against the submitted code, fixed, and covered with 16 new tests (including scorer-acceptance checks against the benchmark's real rule classes) the same day. Both submitted runs were then replayed through the fixed emission **on identical stored model output**—no new inference, so any score change is caused by the emission code alone. The replay's validity was checked directly: re-scoring the stored markdown reproduced the stored per-document metrics with zero mismatches on every rule-scored split of both runs, and the fixed provider port was verified byte-identical to the fixed emission on all 4,156 documents.

Table 5: Effect of fixing our own emission defects, re-scored on identical model output (mean of both runs).

Dimension	As submitted	Fixed	Δ
Tables	86.10	86.10	0.00
Charts	65.33	65.19	-0.14
Content Faithfulness	87.34	87.37	+0.03
Semantic Formatting	71.68	70.64	-1.04
Visual Grounding	74.14	74.14	0.00
Overall	76.92	76.69	-0.23

Semantic Formatting drops 1.04 points once corruption no longer earns credit—consistent in sign and locus with the maintainer's own re-run. Tables is provably unchanged (the extracted table set is identical on every document whose markdown changed), and Visual Grounding is computed from the element payload, which emission never touches. The corrected Overall, 76.69, still clears the preregistered top-tier threshold of 76.36, so the ranking claim survives the correction with a smaller margin (0.33, previously 0.56); the same-environment improvement becomes +4.04 (previously +4.30/+4.24 per run). All published artifacts—leaderboard row, model card, results file, and this paper—carry the corrected numbers with dated correction notes.

6 Limitations

- **Two runs, no confidence intervals.** The reported score is the mean of $N=2$ full-corpus runs (spread 0.06 Overall, maximum per-dimension 0.12). The agreement is tight, but no formal uncertainty estimate accompanies it, and the 0.33 margin over the base model's published row is small.
- **The emission layer's contribution is not isolated from the weights.** The +4.04 same-environment gain combines the fine-tuned weights and the three emission patches. A clean decomposition—base-model output through our emission, and our model's output through the baseline emission—can be computed entirely by replay on stored outputs at zero GPU cost; it is pending and will be published in the reproduction repository. The patch-only replay figure of 74.64 predates the Section 5.2 correction and is likewise pending re-measurement.
- **The insurance-subset score of our model is pending re-measurement.** Our model's insurance-subset figure was measured before the emission correction, so we do not report it here; the subset scripts re-run mechanically once the corrected reports are in place. The baseline's insurance-vs-other gap (84.18 vs 87.91, Section 2.2) is unaffected, as both sides of that comparison come from the same baseline run.
- **The cross-environment comparison is unresolved.** We measure the base pipeline at 72.65 where its published row says 76.36, and could not explain the difference (Section 3.1). If the maintainers' environment re-runs our fixed pipeline and lands below ours, their numbers govern—the pull request states this explicitly.
- **Benchmark-specific supervision.** The training data targets exactly the styling behaviors ParseBench scores (bold, strikethrough, super/subscript). We built it from real filings and exact-by-construction renders rather than from benchmark annotations, but transfer beyond this benchmark's rule set is unmeasured.

- **Not a production system.** Per the preregistration, this work is for publication only. No claim is made that this model is used in, or was built for, a deployed underwriting pipeline.

7 Reproducibility

Everything required to re-derive the numbers in this paper is public:

- **Reproduction repository**^[5]: training loop, data generators, EDGAR harvester, evaluation harness, replay scripts, per-iteration logs, the preregistration with its dated amendments, and the defect-response report for the Section 5.2 correction.
- **Model weights**^[4]: LoRA adapter and merged weights at `florin-inc/florin-parser-nano`, AGPL-3.0 (inherited from the base model), with full attribution to KoreaDeepLearning, the results file, and the model card carrying the dated correction note.
- **Benchmark entry**^[6]: PR #99 to the benchmark repository contains the provider code (the declared emission layer), the leaderboard row, and the review thread of the correction episode; issues #100–#104 contain the defect reports with file and line references.
- **Instrument integrity**: no file under the upstream benchmark checkout was modified; all changes live in our own provider module. The replay harness reproduces the baseline's stored scores exactly (zero per-document mismatches on every rule-scored split), which is what makes the correction episode's emission-only attribution possible.

References

1. ParseBench (LlamaIndex): <https://parsebench.ai>; dataset: <https://huggingface.co/datasets/llamaindex/ParseBench>
2. ParseBench benchmark repository: <https://github.com/run-llama/ParseBench>
3. Base model, KDL-Frontier-Parser-nano (KoreaDeepLearning, 1.2B, AGPL-3.0): <https://huggingface.co/KDLAI/KDL-Frontier-Parser-nano>
4. florin-parser-nano weights (adapter + merged, AGPL-3.0): <https://huggingface.co/florin-inc/florin-parser-nano>
5. Reproduction repository and preregistration: <https://github.com/ammoman21/parsebench-open-weight-sota>
6. Submission and review thread: <https://github.com/run-llama/ParseBench/pull/99>
7. Defect reports: <https://github.com/run-llama/ParseBench/issues/100> through <https://github.com/run-llama/ParseBench/issues/104>
8. SEC EDGAR (U.S. Securities and Exchange Commission filing system): <https://www.sec.gov/edgar>
9. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," 2021. <https://arxiv.org/abs/2106.09685>
10. SERFF, the NAIC System for Electronic Rate and Form Filing: <https://www.serff.com>